

后者要根据数据字典中记载的各种信息,按照一定的优化策略选择一个系统认为“较好”的存取方案,并把选中的方案描述出来。

综上所述,将数据操纵语言语句转换成一串可执行的存取动作这一过程称为一个逐步束缚(binding)的过程。它将数据操纵语言高级的描述型语句(集合操作)转换为系统内部低级的单元组操作,并和具体的数据结构、存取路径、存储结构等结合起来,构成一串确定的存取动作。

13.4 数据存取层

数据存取层介于语言处理层和数据存储层之间。它向上提供单元组接口,即导航式的一次一个元组的存取操作;向下则以系统缓冲区的存储器接口作为实现基础,其接口关系如图 13.5 所示。

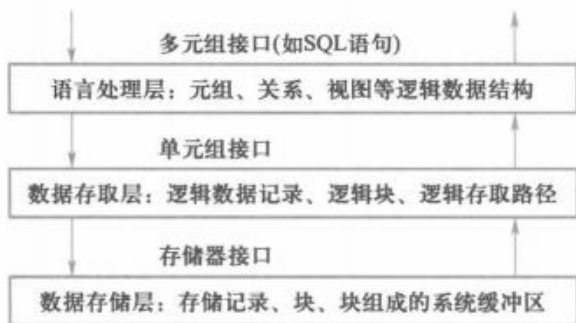


图 13.5 数据存取层及其上下接口关系

图 13.5 中给出了每个层次中操作对象的数据结构。例如,数据存取层所涉及的主要数据结构为逻辑数据记录、逻辑块、逻辑存取路径。

数据存取层的任务主要包括以下几个方面。

- ① 提供一次一个元组的查找、插入、删除、修改等基本操作。
 - ② 提供元组查找所循的存取路径以及对存取路径的维护操作,如对索引记录的查找、插入、删除、修改。若索引是采用 $B+$ 树结构,则应提供 $B+$ 树的建立、查找、插入、删除、修改等功能。
 - ③ 对记录和存取路径的封锁、解锁操作。
 - ④ 对日志文件的登记和读取操作。
 - ⑤ 其他辅助操作,如扫描、合并/排序等,其操作对象有关系、有序表、索引等。
- 为了完成上述功能,通常把数据存取层又划分为若干个功能子系统加以实现。

13.4.1 数据存取层的系统结构

数据存取层包含许多功能,在实际的关系数据库管理系统中由多个功能子系统来完成。

图 13.6 是数据存取层的系统结构，主要包括以下子系统和模块。

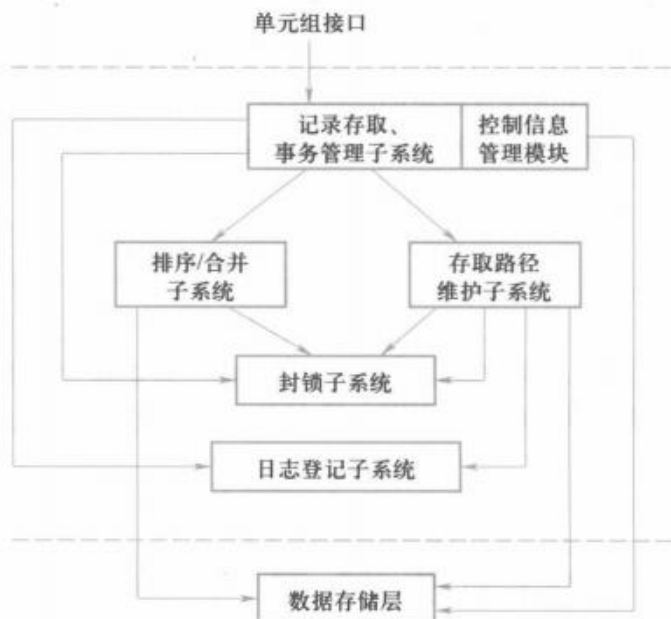


图 13.6 数据存取层的系统结构

- ① 记录存取、事务管理子系统。
- ② 控制信息管理模块。
- ③ 排序/合并子系统。
- ④ 存取路径维护子系统。
- ⑤ 封锁子系统，用以执行并发控制。
- ⑥ 日志登记子系统，用以执行恢复任务。

这些子系统相互配合、紧密联系，构成了一个完整的数据存取层。

13.4.2 数据存取层的功能子系统

数据存取层中有些子系统(如封锁子系统、日志登记子系统等)的功能已在前面的章节中做了介绍，下面只做简要阐述。

1. 记录存取、事务管理子系统

数据存取层不涉及存储分配、存储结构及有关参数，只在数据的逻辑结构上操作，因而可以把各种物理实现形态隐蔽起来。

记录存取子系统提供按某个属性值直接取一个元组和顺序取一个元组的存取原语。这种存取运算是按已选定的某个逻辑存取路径进行的，如某个数据文件或某个索引。这类存取操作的例子有：

- 在某个存取路径上按属性值找元组(FIND)。

- 按相对位置找元组(NEXT, PRIOR, FIRST, LAST)。
- 给某关系增加一个元组(INSERT)。
- 从找到的元组中取某个属性值(GET)。
- 从某关系中删去一个元组(DELETE)。
- 把某修改完的元组写回关系中(REPLACE)。

事务管理子系统提供定义和控制事务的操作。事务管理的基本操作有：

- 定义事务开始(BEGIN TRANSACTION)。
- 事务提交(COMMIT)。
- 事务回滚(ROLLBACK)。

事务管理子系统提供的这些操作将登记进日志文件中。

2. 日志登记子系统

日志登记子系统和事务管理子系统紧密配合，完成关系数据库管理系统对事务和数据库的恢复任务。它把事务的开始、回滚、提交，对元组的插入、删除、修改，以及对索引记录的插入、删除、修改等每一个操作作为一个日志记录存入日志文件中，当事务或系统软硬件发生故障时利用日志文件执行恢复。与日志文件有关的主要操作有：

- 写日志记录(WRITELOG)。
- 读日志记录(READLOG)。
- 扫描日志文件(SCANLOG)。
- 撤销尚未结束的事务(UNDO)。
- 重做已经结束的事务(REDO)。

3. 控制信息管理模块

该模块利用内存中专门的数据区登记不同记录类型以及不同存取路径的说明信息(取自数据字典)和控制信息，这些信息是存取元组和管理事务的依据。控制信息管理模块和记录存取、事务管理子系统一起保证事务的正常运行。

该模块提供对数据字典中说明信息的读取、增加、删除和修改操作。

4. 排序/合并子系统

在语言处理层中，描述性语言表达的集合级操作被转换成一系列对数据存取层所提供的存取原语的调用。为了得到用户所要求的有序输出，同时为了加速关系运算(如自然连接)的中间步骤，常常需要对关系元组重新排序。这一工作由排序/合并子系统来完成。下面列举排序操作的若干主要用途。

(1) 输出有序结果

例如，用户提出如下查询要求：

```
SELECT Eno,Salary
FROM EMP
ORDER BY Salary DESC;
```

若 EMP 表上的 Salary 属性已建有索引,则可以顺序扫描索引获得要求的输出。若 Salary 上没有索引,则必须对 EMP 表按 Salary 的属性值降序排序,以得到所要的结果。

(2) 数据预处理

对于并、交、差、分组聚集、连接、取消重复值、属于、不属于等关系运算,当参与运算的关系无法全部放入内存时,先对其进行排序预处理,再在有序表上执行相应操作,这种做法是降低处理代价的常用手段之一。它可以将操作代价由 $O(n^2)$ 数量级降至 $O(n\log_2 n)$ 数量级。

(3) 支持动态建立索引结构

B+树是数据库中常用的索引结构。B+树的叶页索引记录形式为(码值,TID),其中 TID 为元组标识符。TID 可用元组逻辑记录号、主码值或数据块号加位移等来表示。索引记录在 B+树的叶页上是顺序存储的,因此在初建 B+树索引时首先要对(码值,TID)排序。

(4) 减少数据块的存取次数

通过 B+树索引存取元组时,首先得到(码值,TID)集合,然后根据 TID 存取相应的元组。当 TID 是用数据块号加位移来表示时,可以首先对 TID 排序,使相同或临近块号的 TID 聚集在一起,然后按数据块号顺序存取物理数据块,避免无序状态下重复读块的情况,减少数据块的存取次数。

排序操作的用途还有很多,这里就不一一列举了。

由此可见,排序操作是记录存取系统和存取路径维护子系统都要经常调用的操作。它对提高系统效率具有关键的作用。因此,排序子系统的设计十分重要,应采用高效的外排序算法(因为排序的数据量很大,所以要使用外排序算法)。

5. 存取路径维护子系统

对数据执行插入、删除、修改操作的同时,要对相应的存取路径进行维护。例如,若用 B+树索引作为存取路径,则对元组进行插入、删除、修改操作时要对该表上已建立的所有 B+树索引进行动态维护,插入、删除相应的索引项;否则,就会造成 B+树索引与数据库表的不一致,当再通过 B+树索引结构存取元组时便会导致操作失败或产生错误结果。

6. 封锁子系统

封锁子系统完成并发控制功能。有关封锁的概念和技术,包括封锁的类型、相容矩阵、死锁处理、可串行性准则、两段锁协议等已在第12章详细讨论,这里只说明以下两点:

① 在操作系统中也有并发控制问题,其实现并发控制的方法通常也采用封锁技术。数据库管理系统的封锁技术与操作系统的封锁技术相比(见表13.1所示),内容更加丰富,技术更加复杂。

② 数据库管理系统中封锁子系统设计的难点不仅在于技术复杂,而且在于其实现手段依赖于操作系统提供的环境。如锁表的设计,由于锁表必须能为多个进程共享,能动态建立和释放,因此锁表的设计就随操作系统环境而异。它是封锁子系统设计的关键。

表 13.1 操作系统和数据库管理系统封锁技术的比较

比较项	操作系统	数据库管理系统
封锁对象	单一, 系统资源(包括 CPU、设备、表格等)	多样, 数据库中各种数据对象(包括用户数据、索引(存取路径)、数据字典等)
封锁对象的状态	静态、确定; 各种封锁对象在封锁表中占有一项; 封锁对象数不变	动态, 不确定; 封锁对象动态改变, 常常在执行前不能确定; 一个封锁对象只有当封锁时才在封锁表中占据一项
封锁粒度	不变, 由于封锁对象单一、固定, 封锁粒度不会改变	可变, 封锁可施加到或大或小的数据单位上, 封锁粒度可以是整个数据库、记录或字段
封锁类型	单一, 排他型锁	多样, 一般有共享型锁(S 锁)、排他型锁(X 锁)或其他类型的封锁, 随系统而异

13.5 缓冲区管理

数据存取层的下面是数据存储层。该层的主要功能是存储管理, 包括缓冲区(buffer)管理、内外存交换、外存管理等, 其中缓冲区管理是十分重要的。因此, 本节主要介绍有关缓冲区管理的内容。数据存储层向数据存取层提供的接口是由定长页面组成的系统缓冲区。

系统缓冲区的设立出于两方面的原因: 一是它把数据存储层以上各系统成分与实际的外存设备隔离, 外存设备的变更不会影响其他系统成分, 使关系数据库管理系统具有设备独立性; 二是系统缓冲区设置在内存或虚存, 可以提高存取效率。

关系数据库管理系统利用系统缓冲区缓存数据, 当数据存取层需要读取数据时, 记录存取子系统首先到系统缓冲区中查找。只有当缓冲区中不存在该数据时才真正从外存读入该数据所在的页面。当数据存取层写回一个元组到数据库中时, 记录存取子系统并不把它立即写回外存, 而是仅对该元组所在的缓冲区页面做一标志, 表示可以释放。只有当该用户事务结束或缓冲区已满而需要调入新的外存页面时, 才按一定的淘汰策略把缓冲区中已有释放标志的页面写回外存。这样可以减少内外存交换的次数, 提高存取效率。

系统缓冲区可由内存或虚存组成。由于内存空间紧张, 缓冲区的大小、缓冲区内内存和虚存部分的比例要精心设计, 针对不同应用和环境按一定的模型进行调整, 既不能让缓冲区占据太大的内存空间, 也不能因其空间太小而频频缺页、调页, 造成“抖动”, 影响效率。

缓冲区包含控制信息和若干定长页面。缓冲区管理模块向上层提供的操作是缓冲区的读(READBUF)、写(WRITEBUF)。缓冲区内部的管理操作有查找页、申请页、淘汰页。缓冲区管理调用操作系统的操作有读(READ)、写(WRITE)。图 13.7 给出了数据库缓冲区及向上下层提供的操作示意。以读操作为例, 缓冲区管理的大致过程如图 13.8 所示。可以看到缓冲区管理中的主要算法是淘汰算法和查找算法。操作系统中有许多淘汰算法可以借鉴, 如 FIFO(先进先出)算法、LRU(最近最少使用)算法以及它们的各种改进算法。查找算法用来